



Simulation des Energiebedarfs eines E-Bikes anhand von GPS-Streckendaten

Abschlussprojekt 2026

Programmieren I

Autor(en)
Matrikelnummer
Kohorte
Lektor

TODO: Vorname Nachname
TODO: Matrikelnummer
Abschlussprojekt 2026
TODO: Lektor

Inhaltsverzeichnis

Zusammenfassung	1
1 Einleitung	2
1.1 Beitrag des Projekts	2
1.2 Dokumentation im Bericht	3
2 Anforderungen	4
2.1 Implementierte Kernfunktionen	4
2.2 Erweiterungen gegenueber einer Minimalversion	4
2.3 Nicht implementierte oder bewusst ausgesparte Funktionen	5
2.4 Technische Randbedingungen	5
3 Projektstruktur	6
3.1 Uebersicht der Komponenten	6
3.2 Zentrale Klassen und Funktionen	7
3.3 Dateien und Ausgaben	7
4 Softwarearchitektur und OOP	8
4.1 Objektorientierte Struktur	8
4.2 Klassendiagramm	8
4.3 Daten- und Kontrolltrennung	8
5 GPS- und Kinematikberechnung	10
5.1 Zeit und Distanz	10
5.2 Steigung, Kurs und Kompassrichtung	10
5.3 Beschleunigung	11
5.4 Wichtige DataFrame-Spalten	11
6 Geschwindigkeitsglaettung	12
6.1 Konfigurationsparameter	12
6.2 Ablauf der Glättung	12
6.3 Mathematische Beschreibung	13
6.4 Aktive und passive Spalten	13

7	Physikalisches E-Bike-Modell	14
7.1	Einzelkraefte	14
7.2	Gesamtkraft und Leistung	15
7.3	Parameter	15
8	Motor- und Akkumodelle	16
8.1	Motor	16
8.2	Abstrakte Akku-Schnittstelle	16
8.3	Gemeinsame Akku-Funktionalitaet	16
8.4	LiPo und NMC	17
8.5	Bremswiderstand und Rekuperation	17
9	Simulationsablauf	18
9.1	Datenfluss	18
9.2	Ablauf in <code>main.py</code>	19
9.3	Uebergabene Daten	19
10	Visualisierung und Ausgaben	20
10.1	HTML-Karten	20
10.2	PNG-Diagramme	20
10.3	Logdatei und Verzeichnisse	23
11	Tests und Validierung	24
11.1	Ausgefuehrter Testbefehl	24
11.2	Ergebnis	24
11.3	Interpretation	24
12	Ergebnisse	26
12.1	Streckenkennwerte	26
12.2	Vergleich der Glättung	26
12.3	Interpretation	27
13	Grenzen und Ausblick	28
13.1	Aktuell implementiert	28
13.2	Modellannahmen	28
13.3	Moegliche Erweiterungen	28
13.4	Einordnung	28
14	Fazit	30

Abbildungsverzeichnis

4.1	UML-ähnliche Sicht auf die wesentlichen Klassen und Beziehungen	9
9.1	Programmfluss von den Eingabedaten bis zu den Ausgaben	18
10.1	Hoehenprofil der betrachteten Route	20
10.2	Zeitverlauf der Simulationsgroessen fuer den LiPo-Lauf	21
10.3	Vergleich des Ladezustands beider Akkutypen	22
10.4	Vergleich von Roh- und geglaetteter Geschwindigkeit	22

Tabellenverzeichnis

3.1	Wesentliche Komponenten des Repositories und ihre Verantwortung	6
3.2	Ausgewählte Klassen und ihre öffentlichen Methoden	7
6.1	Parameter der Geschwindigkeitsglättung	12
8.1	Gemeinsamkeiten und Unterschiede der beiden Akkupacks	17
11.1	Abgedeckte Testbereiche	25
12.1	Ermittelte Kennzahlen des aktuellen Datensatzes	26
12.2	Vergleich ohne und mit Glättung	27

Zusammenfassung

Das Abschlussprojekt implementiert eine datengetriebene E-Bike-Simulation auf Basis realer GPS-Streckendaten. Aus einer semikolonseparierten CSV-Datei werden Zeit, Position, Hoehe und Temperatur eingelesen und in Strecke, Geschwindigkeit, Beschleunigung, Steigung und Kurs umgerechnet. Darauf aufbauend berechnet das Programm die fuer die Fahrt benoetigte Antriebskraft, Leistung, das Raddrehmoment und den Motorstrom. Eine zeitbasierte Glättung der Geschwindigkeit kann optional aktiviert werden, um GPS-Rauschen und starke Beschleunigungsspitzen zu reduzieren. Die Simulation fuehrt ein LiPo- und ein NMC-Akkupack mit derselben Fahrstrecke durch und speichert den Verlauf von Leistung, Drehmoment, Strom, Spannung und Ladezustand. Fuer den vorliegenden Datensatz ergibt sich eine Strecke von 94.27 km, eine Fahrzeit von 4.55 h und ein End-SoC von 0 % fuer beide Akkutypen, weil der Akku im Verlauf der Fahrt leer wird.

Kapitel 1

Einleitung

Das Abschlussprojekt untersucht die Frage, wie viel Energie ein E-Bike auf einer real gefahrenen Route benötigt. Ausgangspunkt ist keine synthetische Lastkurve, sondern eine GPS-Aufzeichnung mit Zeitstempeln, Position, Höhe und Temperatur. Dadurch entsteht ein datengetriebenes Modell, das die reale Topografie und das Fahrverhalten der Strecke direkt in die Simulation übernimmt.

Die Motivation für einen solchen Ansatz liegt in der Kopplung mehrerer physikalischer Einflüsse. Die benötigte Antriebskraft hängt nicht nur von der Geschwindigkeit ab, sondern auch von der Beschleunigung, der Steigung, der Fahrmasse und dem Luftwiderstand. Gerade bei einer Route mit wechselnder Höhe und vielen kurzen Messabständen kann GPS-Rauschen zu stark schwankenden Geschwindigkeiten führen. Das Projekt versucht deshalb, die Messdaten aufzubereiten, die Fahrwiderstände zu bestimmen und darauf aufbauend zwei unterschiedliche Akkutechnologien zu vergleichen.

Das Projektziel ist klar abgegrenzt: Es wird kein vollständiges Fahrdynamikmodell mit Fahrerleistung, Wind, Antriebsregelung oder Motorwirkungsgrad simuliert. Stattdessen berechnet die Software aus den GPS-Daten die für die Strecke notwendige mechanische Leistung und speist diesen Lastverlauf in ein LiPo- und ein NMC-Akkupack ein. Die zentrale Forschungsfrage lautet damit nicht, wie das Fahrzeug vorwärts simuliert wird, sondern wie sich eine gegebene Fahrt energetisch auf unterschiedliche Speichertechnologien auswirkt.

1.1 Beitrag des Projekts

Das Programm verbindet Datenverarbeitung, Physik und Visualisierung in einer einzigen Pipeline. Die GPS-Rohdaten werden eingelesen, in Kinematikgrößen umgerechnet, optional geglättet und anschliessend durch ein E-Bike-Modell und zwei Akku-Varianten verarbeitet. Die Ergebnisse werden sowohl numerisch als auch grafisch ausgegeben: als HTML-Karten, als PNG-Diagramme und als Logdatei. Die technische Dokumentation orientiert sich an der verwendeten Klasse [1], während das hier beschriebene Programm selbst im Repository des Abschlussprojekts liegt [2].

1.2 Dokumentation im Bericht

Die folgenden Kapitel beschreiben zunächst die Anforderungen und die Repository-Struktur, dann die objektorientierte Architektur, die GPS- und Kinematikberechnung, die Geschwindigkeitsglat-
tung, das physikalische Modell, die Akku- und Motormodelle, den Simulationsablauf sowie die er-
zeugten Ausgaben. Abschliessend werden die Tests, die realen Simulationsergebnisse und die Gren-
zen des Modells diskutiert.

Kapitel 2

Anforderungen

Das aktuelle Projekt implementiert die im Quellcode nachweisbaren Kernfunktionen einer routen-basierten E-Bike-Simulation. Die Anforderungen lassen sich in bereits umgesetzte Kernfunktionen, sichtbare Erweiterungen und naheliegende Folgearbeiten trennen.

2.1 Implementierte Kernfunktionen

- Einlesen einer GPS-CSV-Datei mit den Spalten `time`, `lat`, `lon`, `ele` und `temperature`.
- Berechnung von Zeitdifferenzen, Distanz, Geschwindigkeit, Steigung, Kurswinkel, Kompass-richtung und Beschleunigung.
- Optionale Geschwindigkeitsglaettung mit konfigurierbarer Zeitbasis.
- Berechnung der Antriebskraft, mechanischen Leistung, des Raddrehmoments und des Motorstroms.
- Simulation des Ladezustands, der Klemmenspannung und der temperaturabhaengigen Akkuparameter fuer LiPo und NMC.
- Erzeugung von HTML-Karten, PNG-Diagrammen und einer Logdatei.

2.2 Erweiterungen gegenueber einer Minimalversion

Das Projekt geht ueber eine reine Streckenauswertung hinaus. Die GPS-Daten werden nicht nur gelesen, sondern in eine zeitlich konsistente Kinematik ueberfuehrt. Ausserdem verwendet das Programm eine abstrakte Akkobasis, damit der Simulator mit mehreren Akkutypen arbeiten kann. Die Luftdichte wird nicht konstant angesetzt, sondern aus Hoehe und Temperatur abgeschaezt, und ein Bremswiderstandsmodell faengt Ueberschuesse beim Laden ab.

2.3 Nicht implementierte oder bewusst ausgesparte Funktionen

Mehrere inhaltlich naheliegende Erweiterungen sind im aktuellen Code nicht enthalten und werden daher im Bericht auch nicht behauptet: direkte Fahrerleistung, Windmodell, Motorwirkungsgrad, Rueckspeisung in ein reales Ladenetz, Sensordaten von Leistung oder Drehmoment sowie eine aktive Regelung des Antriebs. Ebenso gibt es keine Pauschalbegrenzung der Geschwindigkeit oder Beschleunigung; die Verarbeitung arbeitet ausschliesslich daten- und segmentbasiert.

2.4 Technische Randbedingungen

Die Software ist fuer einen lokalen Python-Workflow mit virtueller Umgebung ausgelegt. Installiert werden die in `requirements.txt` genannten Pakete sowie LuaLaTeX. Die Ausgabedateien werden im Verzeichnis `output/` erzeugt; der Bericht selbst verwendet davon getrennte Kopien der wichtigsten Plot-Dateien im Verzeichnis `report/figures/generated/`.

Kapitel 3

Projektstruktur

Das Repository ist flach organisiert und trennt die Fachlogik in kleine, klar benannte Module. Das erleichtert sowohl die Simulation als auch die Wartung, weil GPS-Verarbeitung, Fahrzeugphysik, Motorlogik, Akkuverhalten und Visualisierung getrennt entwickelt werden koennen.

3.1 Uebersicht der Komponenten

Tabelle 3.1: Wesentliche Komponenten des Repositories und ihre Verantwortung

Komponente	Verantwortung
main.py	Programmstart, Logging, Laden der Konfiguration, Erzeugen von Track, Fahrzeug, Motor und Akkus sowie Start der Simulation.
gps_track.py	Einlesen der GPS-CSV, Berechnung von Distanz, Geschwindigkeit, Steigung, Kurs, Kompassrichtung und Beschleunigung sowie Kartenerzeugung.
speed_smoothing.py	Laden der Glättungskonfiguration, Segmentierung und zeitbasierte Glättung der Geschwindigkeitskurve.
ebike.py	Fahrzeugphysik mit Luft-, Hang-, Roll- und Beschleunigungsanteilen.
motor.py	Umrechnung von Drehmoment in Motorstrom ueber die Motorkonstante.
ebike_simulator.py	Schrittweise Simulation der Fahrt und Aufzeichnung der Ergebnisprofile.
battery_base.py	Abstrakte Schnittstelle fuer Batteriemodelle.
battery_pack.py	Gemeinsame Akku-Funktionalitaet, SoC-Integration, Temperaturkorrektur und Begrenzung.
lipo_battery.py	LiPo-spezifische OCV-Kennlinie und Packparametrisierung.
nmc_battery.py	NMC-spezifische OCV-Kennlinie und Packparametrisierung.
akkutemperatur.py	Temperaturabhängige Korrektur von Innenwiderstand und Kapazitaet.
luftdichte.py	Schaetzung der Luftdichte aus Hoehe und Temperatur.
bremswiderstand.py	Umrechnung von ueberschuessigem Strom in dissipierte Leistung.
plot_utils.py	Matplotlib-Plots fuer Hoehenprofil, Zeitverlaeufe und SoC-Vergleich.
data/	Eingabedaten und Konfigurationsdateien.
tests/	Unittest-Suite fuer Glättung, Kinematik und Simulator-Kompatibilitaet.
output/	Laufzeit-Ausgaben des Programms.

3.2 Zentrale Klassen und Funktionen

Tabelle 3.2: Ausgewählte Klassen und ihre öffentlichen Methoden

Klasse oder Funktion	Öffentliche Methoden und Signaturen
GPSTrack	GPSTrack(pfad: str Path, smoothing_config: SpeedSmoothingConfig None = None), gesamtstrecke_km(), durchschnittsgeschwindigkeit_kmh(), gesamtzeit_s(), hoehenmeter_anstieg(), hoehenmeter_abstieg(), haufigste_kompassrichtung(), kennzahlen_ausgeben(), smoothing_kennzahlen(), karte_erstellen(...)
SpeedSmoothingConfig	from_yaml(path), Attribut max_reasonable_speed_ms
EBike	from_yaml(pfad), luftwiderstand_N(...), hangabtriebskraft_N(...), beschleunigungskraft_N(...), rollwiderstand_N(...), antriebskraft_N(...), leistung_W(...), drehmoment_Nm(...), punkt_auswerten(...)
Motor	Motor(motorkonstante_Nm_A: float = 1.5), get_current_draw(torque_Nm: float)
BatteryBase	apply_current(current: float, duration: float), voltage(current: float = 0.0)
BatteryPack	set_temperatur(temperature_c: float None = None), apply_current(...), voltage(...), is_empty(), is_full()
LiPoBatteryPack	_ocv(soc: float) als Uebersteuerung der Leerlaufspannungskennlinie
NMCBatteryPack	_ocv(soc: float) als Uebersteuerung der Leerlaufspannungskennlinie
EBikeSimulator	simulate(), maximaleleistung_W(), bremswiderstand_energie_Wh(), endladezustand_prozent(), zusammenfassung_ausgeben()

3.3 Dateien und Ausgaben

Die wichtigsten Eingabedateien liegen unter data/. Die Ergebnisdateien werden zur Laufzeit unter output/ abgelegt, wobei Unterordner fuer Plots automatisch erzeugt werden. Fuer den Bericht wurden die reproduzierbaren PNG-Dateien nach report/figures/generated/ kopiert, damit der PDF-Build ohne Abhaengigkeit vom letzten Programmlauf funktioniert.

Kapitel 4

Softwarearchitektur und OOP

Die Architektur des Projekts folgt einer klaren Trennung zwischen Datenquelle, physikalischem Modell, Energieversorgung und Visualisierung. Die Klassen kooperieren ueber Assoziationen und Vererbung, aber die fachliche Verantwortung bleibt in getrennten Modulen gebuendelt.

4.1 Objektorientierte Struktur

`BatteryBase` definiert die abstrakte Schnittstelle fuer alle Akku-Modelle. `BatteryPack` liefert die gemeinsame Implementierung mit SoC-Integration und Temperaturkorrektur. `LiPoBatteryPack` und `NMCBatteryPack` spezialisieren nur die Leerlaufspannungskennlinie sowie die physikalischen Packparameter. Dadurch kann `EBikeSimulator` den Akku polymorph verwenden, ohne den konkreten Typ kennen zu muessen.

`GPSTrack` ist die Datenquelle fuer die Simulation. `EBike` modelliert die Fahrphysik, `Motor` die Umrechnung von Drehmoment in Strom. Der Simulator verbindet diese Objekte und fuehrt die Strecke zeitdiskret aus. So entstehen klare Zustaeendigkeiten: Datenaufbereitung in `GPSTrack`, Physik in `EBike`, elektrische Umrechnung im `Motor`, Energiespeicherung im Akku und Ablaufsteuerung im Simulator.

4.2 Klassendiagramm

Das Diagramm zeigt die im Code sichtbare Struktur: Vererbung nach unten bei den Batterien und Assoziationen vom Simulator zu den vier zentralen Objekten. Die Richtung der Pfeile ist absichtlich einfach gehalten, weil der Schwerpunkt auf der inhaltlichen Architektur liegt, nicht auf einer vollstaendigen UML-Notation.

4.3 Daten- und Kontrolltrennung

Die Datenverarbeitung ist von der Simulation entkoppelt. Das Programm liest die CSV und bereitet sie in `GPSTrack` auf, erzeugt danach ein Objekt des E-Bikes und des Motors und fuehrt dann die Si-

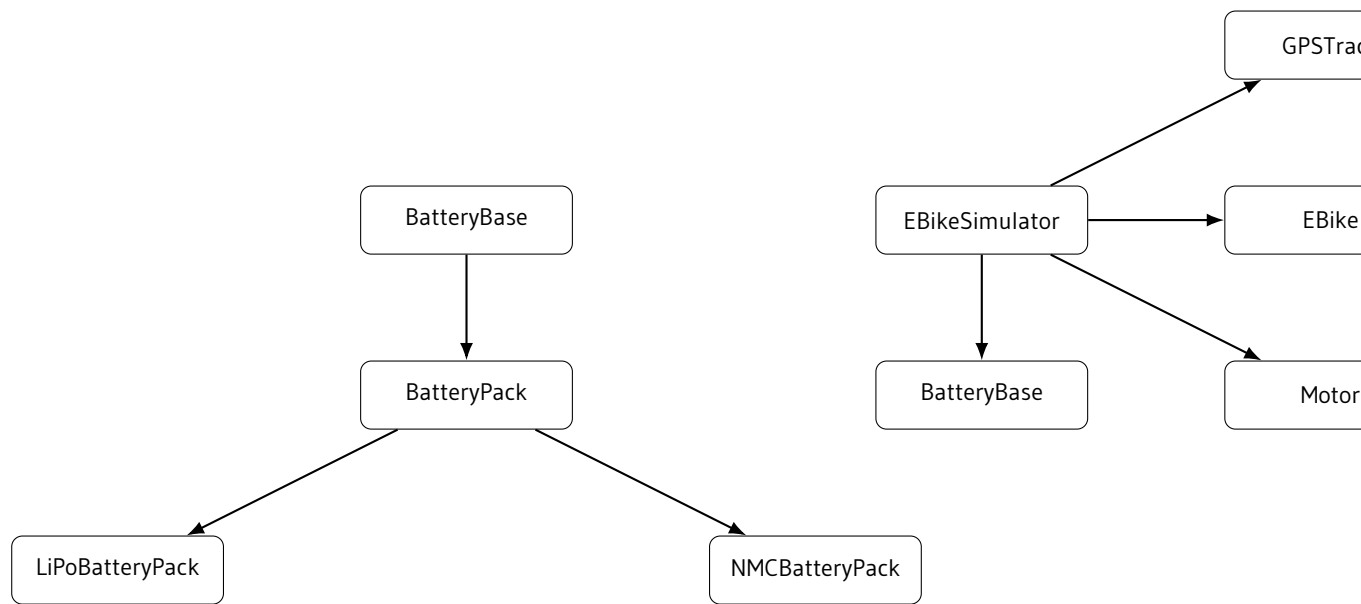


Abbildung 4.1: UML-ähnliche Sicht auf die wesentlichen Klassen und Beziehungen

mulation fuer jeden Akkutyp separat aus. Diagramme und Karten sind rein nachgelagerte Ausgaben; sie beeinflussen das Simulationsmodell nicht.

Kapitel 5

GPS- und Kinematikberechnung

Die GPS-Eingabe liegt als semikolonseparierte CSV vor und enthaelt die Spalten `time`, `lat`, `lon`, `ele` und `temperature`. Die Zeitstempel werden mit `pandas.to_datetime` eingelesen und in UTC interpretiert [3]. Das Projekt setzt voraus, dass die Zeilen bereits in zeitlicher Reihenfolge vorliegen; die Berechnung selbst arbeitet dann nur mit aufeinanderfolgenden Punkten.

5.1 Zeit und Distanz

Fuer zwei aufeinanderfolgende Punkte $i - 1$ und i gilt im Code:

$$\Delta t_i = t_i - t_{i-1} \quad (5.1)$$

$$\Delta s_i = 2R \arcsin(\sqrt{a}) \quad (5.2)$$

mit

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_{i-1}) \cos(\varphi_i) \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (5.3)$$

und $R = 6\,371\,000$ m als mittlerem Erdradius. Diese Haversine-Formel wird direkt in `GPSTrack_distanz_haversine()` umgesetzt.

Aus Distanz und Zeit folgt die Rohgeschwindigkeit:

$$v_i = \frac{\Delta s_i}{\Delta t_i} \quad (5.4)$$

Wenn $\Delta t_i \leq 0$ ist, wird die Rohgeschwindigkeit im Projekt auf 0 m/s gesetzt und das Intervall spaeter als ungueltig markiert. Die Berechnung vermeidet damit Divisionen durch null und erhaelt trotzdem die restlichen Datenpunkte.

5.2 Steigung, Kurs und Kompassrichtung

Die Hoehendifferenz wird als

$$\Delta h_i = h_i - h_{i-1} \quad (5.5)$$

verwendet. Daraus entsteht die Steigung in Grad als

$$\varphi_i = \arctan\left(\frac{\Delta h_i}{\Delta s_i}\right) \quad (5.6)$$

falls $\Delta s_i > 0$ ist, sonst 0. Der Kurswinkel wird aus Breiten- und Laengengrad ueber die im Code implementierte Anfangskurs-Formel bestimmt und anschliessend auf den Bereich $[0, 360)$ normalisiert. Die Kompassrichtung wird daraus auf die acht Hauptrichtungen N, NO, O, SO, S, SW, W und NW abgebildet.

5.3 Beschleunigung

Die Beschleunigung wird nicht aus einer verrauschten Punkt-fuer-Punkt-Differenz gebildet, sondern segmentweise aus der aktiven Geschwindigkeitskurve. Die Funktion `beschleunigung_aus_geschwindigkeit()` verwendet eine robuste Randbehandlung und zentrale Differenzen innerhalb eines Segments. Dadurch werden harte Spruenge ueber grosse Zeitluecken vermieden.

5.4 Wichtige DataFrame-Spalten

Die berechneten Spalten bilden die Grundlage fuer die weiteren Kapitel. Besonders relevant sind:

`dt_s`, `distanz_m`, `steigung_grad`, `kurs_grad`, `kompassrichtung`, `geschwindigkeit_roh_ms`, `geschwindigkeit_gegl`, `geschwindigkeit_ms`, `beschleunigung_roh_ms2`, `beschleunigung_geglaettet_ms2`, `beschleunigung_ms2` sowie die Filterspalten fuer die Glättung. Der Simulator arbeitet spaeter ausschliesslich mit `geschwindigkeit_ms` und `beschleunigung_ms2`.

Kapitel 6

Geschwindigkeitsglaettung

GPS-Daten liefern oft leicht schwankende Positionen, insbesondere bei kurzen Zeitintervallen. Da die Geschwindigkeit im Projekt aus Distanz durch Zeit berechnet wird, erzeugen kleine Zeitdifferenzen schnell hohe Spitzen. Diese Spitzen wirken direkt auf die Beschleunigung und damit auf die benötigte Leistung. Das Projekt bietet deshalb eine zeitbasierte Glättung, die ueber die Konfiguration in `data/speed_smoothing_config.yaml` aktiviert oder deaktiviert wird.

6.1 Konfigurationsparameter

Die aktuelle Konfiguration enthaelt die folgenden Werte:

Tabelle 6.1: Parameter der Geschwindigkeitsglaettung

Parameter	Bedeutung
<code>enabled</code>	Schaltet die Glättung ein oder aus.
<code>min_interval_s</code>	Untere Schranke fuer gueltige Filterstuetzstellen.
<code>max_gap_s</code>	Zeitluecke, ab der ein neues Segment beginnt.
<code>median_window_s</code>	Zeitfenster des zeitbasierten Medianfilters.
<code>time_constant_s</code>	Zeitkonstante der exponentiellen Glättung.
<code>max_reasonable_speed_kmh</code>	Plausibilitaetsgrenze fuer Stuetzstellen.

6.2 Ablauf der Glättung

Der Algorithmus arbeitet segmentweise und fuehrt die folgenden Schritte aus:

1. Rohgeschwindigkeit berechnen.
2. Ungueltige Intervalle erkennen, also $\Delta t \leq 0$.
3. Sehr kurze Intervalle als ungeeignete Stuetzstellen markieren.
4. Grosse Zeitluecken in getrennte Segmente aufteilen.

5. Unplausible Geschwindigkeiten als Ausreisser markieren.
6. Einen zeitbasierten Medianfilter anwenden.
7. Fehlende Werte nur innerhalb des Segments interpolieren.
8. Eine zeitabhaengige exponentielle Glättung vorwaerts ausfuehren.
9. Die Glättung rueckwaerts wiederholen.
10. Beide Verlaeufe mitteln.
11. Die Beschleunigung aus der aktiven Geschwindigkeitskurve berechnen.

6.3 Mathematische Beschreibung

Die zeitabhaengige Glättung verwendet den Faktor

$$\alpha_i = 1 - \exp\left(-\frac{\Delta t_i}{\tau}\right) \quad (6.1)$$

mit der Zeitkonstante τ . Daraus folgt die Rekursion

$$v_{\text{glatt},i} = v_{\text{glatt},i-1} + \alpha_i (v_i - v_{\text{glatt},i-1}) \quad (6.2)$$

Die Rueckwaertsfilterung verwendet dieselbe Formel auf der umgekehrten Zeitachse; das Ergebnis wird anschliessend mit dem Vorwaertsverlauf gemittelt.

Die Eingabekonfiguration wird aus YAML geladen [4]; die Glättungsparameter werden dabei auf Plausibilitaet geprueft, bevor sie in die Segmentierung und Filterung einfließen.

6.4 Aktive und passive Spalten

Das Projekt unterscheidet bewusst zwischen Roh-, geglätteter und aktiver Groesse. `geschwindigkeit_roh_ms` bleibt immer erhalten. `geschwindigkeit_geglaettet_ms` zeigt das Filterergebnis, waehrend `geschwindigkeit_ms` die fuer die Simulation aktive Kurve enthaelt. Dasselbe gilt fuer `beschleunigung_roh_ms2`, `beschleunigung_geglaettet_ms2` und `beschleunigung_ms2`. Es gibt keine pauschale Begrenzung oder das Abschneiden extremer Werte; stattdessen werden nur Filterstützstellen und Segmente intelligent behandelt.

Kapitel 7

Physikalisches E-Bike-Modell

Das Fahrzeugmodell in `ebike.py` berechnet fuer jeden GPS-Punkt die benoetigte Kraft aus Fahrwiderstaenden, Beschleunigung und Steigung. Die Rechnung erfolgt in Fahrtrichtung und nutzt die aus dem GPS-Datensatz abgeleitete Geschwindigkeit und Beschleunigung.

7.1 Einzelkraefte

Der Luftwiderstand folgt im Projekt direkt der bekannten quadratischen Form und entspricht der in der Aerodynamik ueblichen Drag-Gleichung [5]:

$$F_{\text{Luft}} = \frac{1}{2} \rho c_w A v^2 \quad (7.1)$$

Dabei ist ρ die Luftdichte, $c_w A$ die aerodynamische Kennzahl und v die Geschwindigkeit. Fuer die Luftdichte verwendet das Projekt entweder die ueber `luftdichte.py` berechnete aktuelle Dichte oder eine konstante Standarddichte, falls Hoehe und Temperatur nicht nutzbar sind.

Die Hangabtriebskraft lautet

$$F_{\text{Hang}} = mg \sin(\varphi) \quad (7.2)$$

und die Beschleunigungskraft

$$F_{\text{Beschleunigung}} = ma \quad (7.3)$$

mit der Gesamtmasse m , der Erdbeschleunigung g , der Steigung φ und der Beschleunigung a .

Der Rollwiderstand wird im Code als

$$F_{\text{Roll}} = c_R mg \cos(\varphi) \quad (7.4)$$

modelliert. Die verwendete Masse setzt sich aus Fahrer- und Fahrradmasse zusammen.

7.2 Gesamtkraft und Leistung

Die Funktion `EBike.antriebskraft_N()` bildet die Summe

$$F_{\text{Antrieb}} = F_{\text{Luft}} + F_{\text{Hang}} + F_{\text{Beschleunigung}} + F_{\text{Roll}} \quad (7.5)$$

und damit genau die im Quellcode verwendete Kraft. Aus Kraft und Geschwindigkeit folgt die mechanische Leistung

$$P = Fv \quad (7.6)$$

und das Raddrehmoment

$$M = Fr \quad (7.7)$$

mit dem Radradius r .

7.3 Parameter

Die Parameter werden aus `data/bike_config.yaml` geladen. Aktuell verwendet das Projekt eine Fahrer- und Radmasse von zusammen 80 kg, eine aerodynamische Kennzahl von $0,5625 \text{ m}^2$, einen Raddurchmesser von 27 cm und einen Rollwiderstandskoeffizienten von 0,005. Diese Werte bestimmen gemeinsam mit der GPS-Kinematik die Leistungsanforderung entlang der Strecke.

Kapitel 8

Motor- und Akkumodelle

8.1 Motor

Die Klasse `Motor` bildet eine einfache lineare Umrechnung vom Raddrehmoment auf den Strom ab. Verwendet wird die im Projekt explizit geforderte Motorkonstante:

$$I = \frac{M}{k_m} \quad (8.1)$$

Die Motorkonstante ist in N m/A angegeben. Der Motor ist als eigene Klasse modelliert, damit die Beziehung zwischen mechanischer Last und elektrischem Strom explizit und austauschbar bleibt.

8.2 Abstrakte Akku-Schnittstelle

`BatteryBase` definiert nur das Interface. Jede konkrete Akku-Klasse muss mindestens die Aktualisierung des Ladezustands und die Berechnung der Klemmenspannung bereitstellen. Diese Abstraktion ist entscheidend fuer den Polymorphismus im Simulator: Der Code arbeitet auf dem Basistyp, nicht auf einer konkreten Unterklasse.

8.3 Gemeinsame Akku-Funktionalitaet

`BatteryPack` implementiert ein Ersatzschaltbild aus Leerlaufspannung und Innenwiderstand. Der Ladezustand wird ueber die Stromintegration aktualisiert:

$$\text{SoC}_{k+1} = \text{SoC}_k - \frac{I \Delta t}{C_{\text{nom}}} \quad (8.2)$$

Die Nennkapazitaet wird von A h nach A s umgerechnet. Positiver Strom entlaedt den Akku, negativer Strom laedt ihn. Die Klemmenspannung ergibt sich als

$$U = U_{\text{OC}}(\text{SoC}) - R_{\text{int}} I \quad (8.3)$$

und wird temperaturkorrigiert, wenn eine Temperatur gesetzt wurde.

Temperatur beeinflusst zwei Groessen: Der Innenwiderstand steigt bei kaelteren Temperaturen relativ zur Referenz von 25 °C, waehrend die nutzbare Kapazitaet sinkt. Beide Korrekturen sind nach unten begrenzt, damit bei extremen Temperaturen keine unrealistische Extrapolation entsteht.

8.4 LiPo und NMC

LiPo- und NMC-Akkus erben beide von `BatteryPack` und unterscheiden sich nur in der Kennlinie und im Innenwiderstand. Das Projekt verwendet tabellarische Leerlaufspannungskennlinien mit linearer Interpolation [6]. Dadurch laesst sich das Verhalten beider Akkus unter identischem Lastprofil direkt vergleichen.

Tabelle 8.1: Gemeinsamkeiten und Unterschiede der beiden Akkupacks

Aspekt	LiPoBatteryPack	NMCBatteryPack
Vererbung	Erbt von <code>BatteryPack</code>	Erbt von <code>BatteryPack</code>
Serienschaltung	10 Zellen	10 Zellen
Innenwiderstand	8 mΩ pro Zelle, skaliert mit <code>n_parallel</code>	7 mΩ pro Zelle, skaliert mit <code>n_parallel</code>
OCV-Kennlinie	LiPo-Stuetzstellen aus Tabelle	NMC-Stuetzstellen aus Tabelle
Interpolation	<code>numpy.interp</code>	<code>numpy.interp</code>
Verhalten unter Last	Hoehere Leerlaufspannung bei gleichem SoC	Niedrigere Leerlaufspannung bei gleichem SoC

8.5 Bremswiderstand und Rekuperation

Wenn beim Bergabfahren oder Bremsen ein negativer Motorstrom entsteht, steigt der Ladezustand des Akkus. Ueberschreitet dieser Ladevorgang 100 %, wird der zulaessige Strom begrenzt. Der ueberschuessige Strom wird dann ueber den Bremswiderstand in Waerme umgesetzt und in `letzte_dissipierte_leistung_w` gespeichert. Im aktuellen Datensatz tritt dieser Fall nicht auf; die gespeicherte dissipierte Leistung bleibt in beiden Simulationslaeufen bei null.

Kapitel 9

Simulationsablauf

9.1 Datenfluss

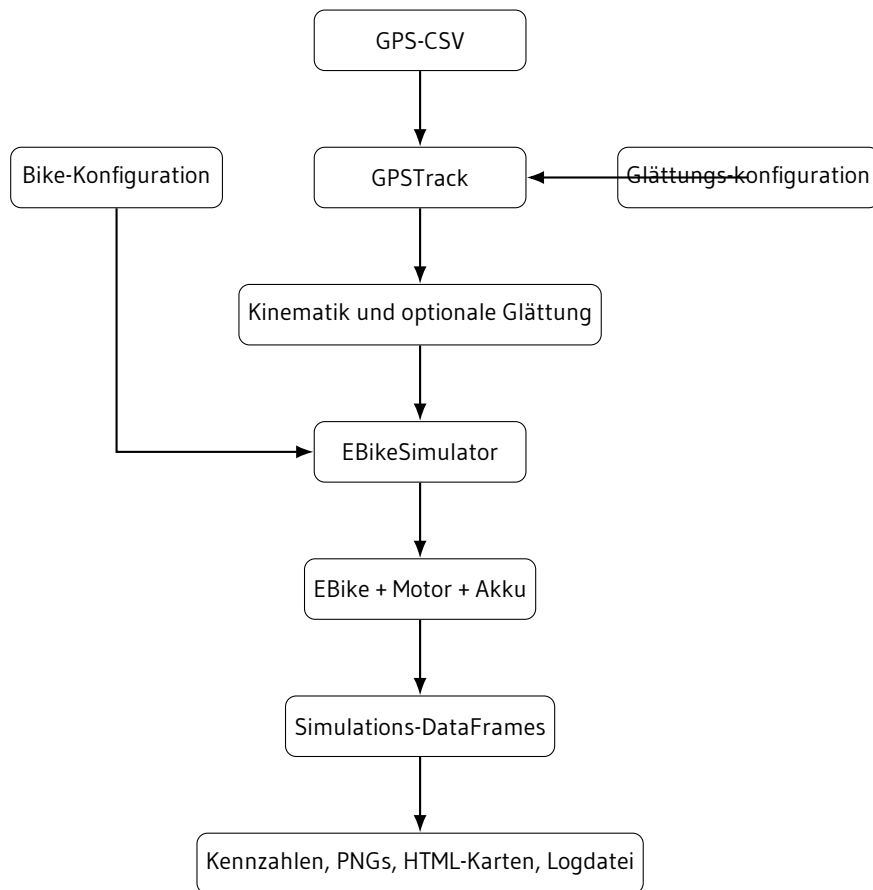


Abbildung 9.1: Programmfluss von den Eingabedaten bis zu den Ausgaben

9.2 Ablauf in main.py

Der Programmstart erzeugt zunaechst die Ausgabeverzeichnisse und initialisiert das Logging. Danach werden die Glättungskonfiguration und der GPS-Track geladen. Anschliessend gibt das Programm die wichtigsten Streckenkennzahlen aus und schreibt eine interaktive Karte mit der Streckenfarbe nach Geschwindigkeit.

Im naechsten Schritt werden E-Bike, Motor und zwei Akkupacks instanziiert. Dann laeuft der Simulator zweimal: einmal fuer LiPo und einmal fuer NMC. Beide Laeufer verwenden dieselbe Track-Datenbasis. Nach jedem Lauf wird eine SoC-Karte erzeugt und das Ergebnis-DataFrame gespeichert. Zum Schluss erstellt `plot_utils.py` die statischen Diagramme.

9.3 Uebergebene Daten

Zwischen den Komponenten werden ausschliesslich DataFrames, einfache Zahlen und konkrete Klassenobjekte uebergeben. GPSTrack liefert die physikalisch aufbereiteten Spalten, EBikeSimulator erweitert diese um Leistungs- und Akkuwerte. Dadurch bleibt die Schnittstelle nachvollziehbar und leicht testbar.

Kapitel 10

Visualisierung und Ausgaben

Das Projekt erzeugt zwei Arten von Ausgaben: interaktive HTML-Karten mit Folium und statische PNG-Diagramme mit Matplotlib. Beide Ausgabearten sind rein nachgelagert und greifen nur auf die bereits berechneten Daten zu.

10.1 HTML-Karten

Die Strecke wird als farbcodierte Linie in `output/karte_strecke.html` gespeichert. Zusätzlich erzeugt der Simulator zwei SoC-Karten fuer die beiden Akkutypen: `output/karte_soc_lipo.html` und `output/karte_soc_nmc.html`. Die Karten werden mit `folium` aufgebaut [7] und koennen im Browser direkt geoeffnet werden. Als Kartenhintergrund wird standardmaessig ein kartenkompatibler Stil verwendet, damit lokale HTML-Dateien ohne Referer-Problem funktionieren.

10.2 PNG-Diagramme

Die statischen Plots liegen unter `output/plot/`. Fuer den Bericht wurden ihre reproduzierbaren Kopien nach `report/figures/generated/` uebernommen und mit `matplotlib` erzeugt [8].

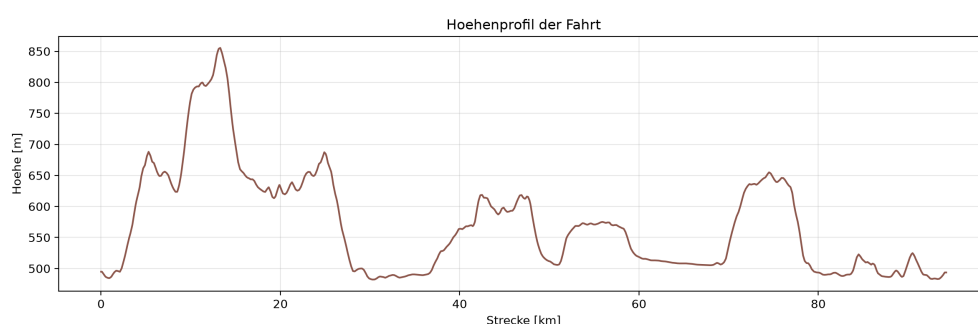


Abbildung 10.1: Hohenprofil der betrachteten Route

Das Höhenprofil in Abbildung 10.1 zeigt die topografischen Wechsel der Strecke. Diese Höhenänderungen sind fuer die spaetere Leistungs- und Akkuabschätzung zentral, weil Steigung und Gefaelle direkt in die Hangabtriebskraft eingehen.

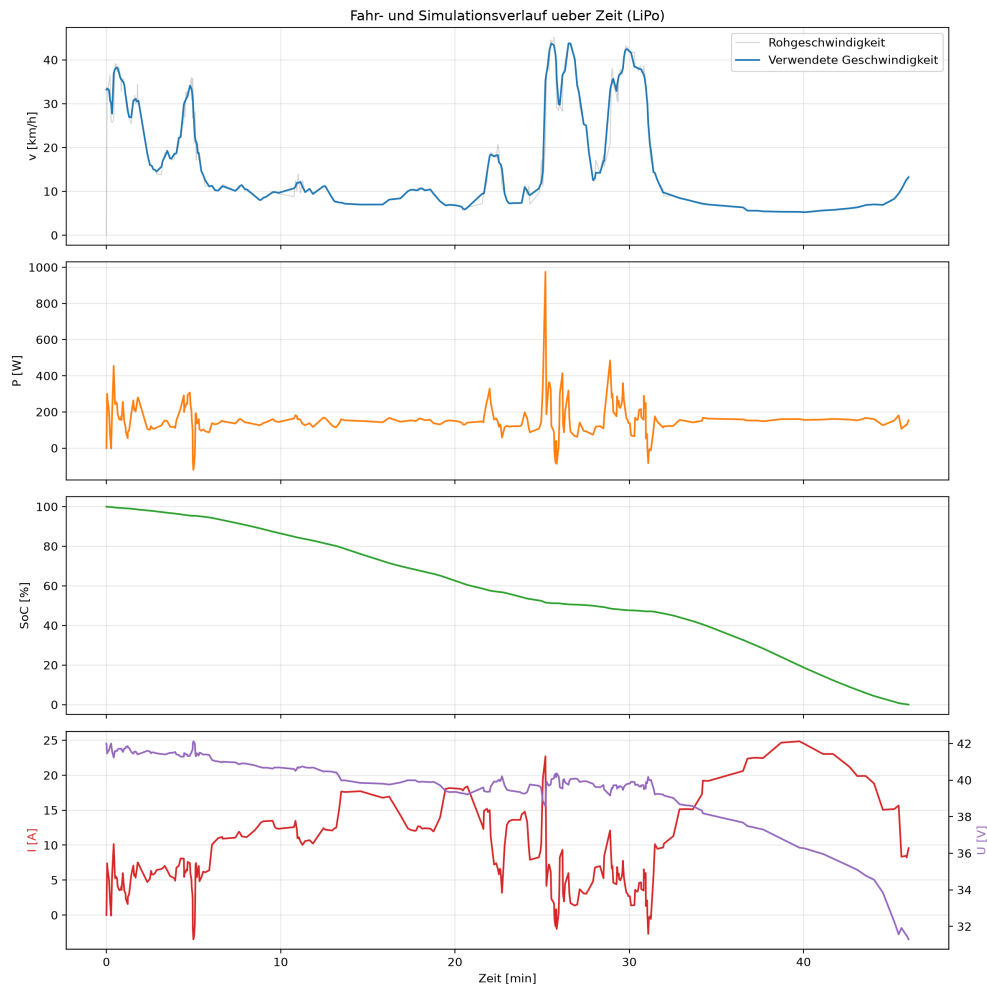


Abbildung 10.2: Zeitverlauf der Simulationsgroessen fuer den LiPo-Lauf

Abbildung 10.2 kombiniert Geschwindigkeit, Leistung, Ladezustand, Strom und Spannung. Man erkennt, dass die Leistungsspitzen und der Stromverlauf direkt von der aktiven Geschwindigkeitskurve abhängen. Die Zeitverlaeufoe fuer NMC sind im Projekt analog erzeugt; die Diagrammdatei liegt ebenfalls im Verzeichnis der generierten Abbildungen.

Der SoC-Vergleich in Abbildung 10.3 zeigt beide Akkus im selben Fahrprofil. Weil die Simulation im aktuellen Datensatz wegen leerem Akku endet, laufen beide Kurven bis auf 0% herunter und erreichen dort ihre Untergrenze.

Die Diagnosegrafik in Abbildung 10.4 zeigt den Filtereffekt direkt. Die geglättete Kurve entfernt

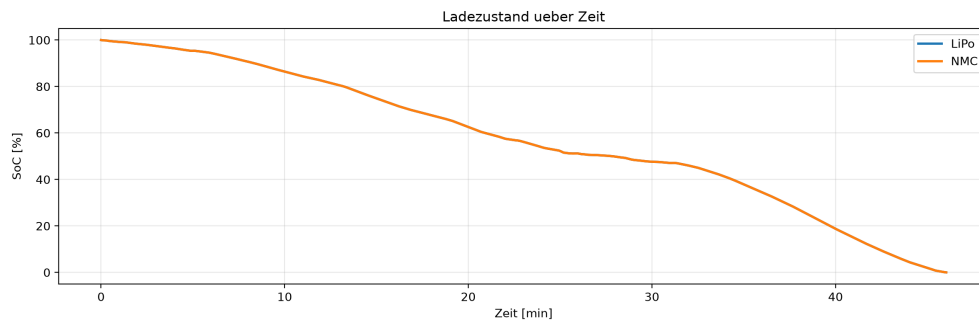


Abbildung 10.3: Vergleich des Ladezustands beider Akkutypen

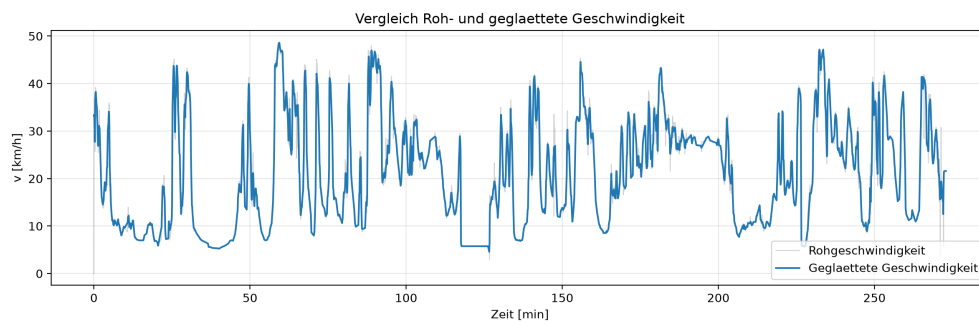


Abbildung 10.4: Vergleich von Roh- und geglaetteter Geschwindigkeit

schnelle, dateninduzierte Zacken, ohne die Strecke selbst oder die maximale Rueckfuehrung auf null zu beschneiden.

10.3 Logdatei und Verzeichnisse

Die Logdatei `output/ebike_simulation.log` spiegelt die Konsolenausgaben mit Zeitstempeln. Das Programm erzeugt alle benoetigten Verzeichnisse beim Start automatisch. Der Bericht selbst verwendet davon getrennte, versionierte Kopien der wichtigsten PNGs.

Kapitel 11

Tests und Validierung

Das Projekt enthielt im aktuellen Stand zwei Testdateien. Sie pruefen die Konfiguration und das Verhalten der Geschwindigkeitsglättung sowie die Integration mit GPSTrack und dem Simulator.

11.1 Ausgefuehrter Testbefehl

Die Tests wurden mit dem folgenden Befehl ausgefuehrt:

```
1 python -m unittest discover -s tests -v
```

11.2 Ergebnis

Die Test-Suite umfasst 14 Einzeltests. Alle 14 Tests sind erfolgreich durchgelaufen. Damit sind die Konfigurationsladung, die Aktivierung und Deaktivierung der Glättung, konstante Geschwindigkeit, Ausreisser, kurze Intervalle, grosse Zeitluecken, unregelmässige Zeitabstaende sowie die Kompatibilitaet mit dem Simulator abgedeckt.

11.3 Interpretation

Die Tests pruefen nicht die gesamte numerische Genauigkeit des Projekts, sondern die erwarteten Verhaltensweisen an den kritischen Schnittstellen. Insbesondere sichern sie ab, dass die Glättung keine pauschale Begrenzung einbaut und dass der Simulator die vom Track bereitgestellten aktiven Spalten ohne Zusatzannahmen weiterverarbeiten kann.

Tabelle 11.1: Abgedeckte Testbereiche

Testbereich	Gepruefte Aussage
Konfigurationsladen	YAML-Werte werden eingelesen, Defaults greifen und unguel-tige Werte fuehren zu Fehlern.
Aktivierte und deaktivierte Glättung	Die aktiven Spalten folgen entweder den Roh- oder den ge-glaetteten Werten.
Konstante Geschwindigkeit	Die Beschleunigung bleibt nahe null.
Einzelner Ausreisser	Ein grosser Geschwindigkeitssprung wird als Filterstuetzstel-le ausgeschlossen.
Kurzes Intervall	Sehr kleine Zeitabstaende werden nicht als gueltige Stuetz-stellen verwendet.
Grosse Zeitluecke	Neue Segmente beginnen an den richtigen Stellen, ohne ueber die Luecke zu interpolieren.
Unregelmässige Intervalle	Der zeitabhaengige exponentielle Faktor reagiert auf unter-schiedliche Zeitabstaende.
Simulator-Kompatibilitaet	EBikeSimulator verarbeitet das Track-DataFrame mit den aktiven Spalten korrekt.

Kapitel 12

Ergebnisse

Die folgenden Kennzahlen wurden aus dem aktuellen Datensatz und dem bestehenden Code abgeleitet. Alle Streckenkennwerte bleiben zwischen den beiden Glättungsläufen gleich, weil sie nur von den GPS-Daten abhängen. Unterschiede entstehen ausschliesslich in Geschwindigkeit, Beschleunigung und den daraus folgenden elektrischen Grössen.

12.1 Streckenkennwerte

Tabelle 12.1: Ermittelte Kennzahlen des aktuellen Datensatzes

Kennwert	Wert
GPS-Punkte	2284
Gesamtdauer	16372.391 s
Gesamtstrecke	94.267 km
Hoehenmeter aufwaerts	1095.895 m
Hoehenmeter abwaerts	1097.146 m
Durchschnittsgeschwindigkeit	20.728 km/h
Maximale Rohgeschwindigkeit	48.772 km/h
Maximale geglaettete Geschwindigkeit	48.584 km/h
Maximale Rohbeschleunigung	2.982 m/s ²
Minimale Rohbeschleunigung	-3.790 m/s ²

12.2 Vergleich der Glättung

Die Strecke selbst bleibt unverändert, aber die Glättung reduziert die Spitzen deutlich. Besonders der Unterschied bei Beschleunigung, Leistung und Motorstrom ist relevant: Ohne Glättung führt ein kurzer Geschwindigkeitssprung zu einer sehr hohen Lastspitze. Mit Glättung wird der Verlauf deutlich ruhiger, während die Strecke und die mittlere Geschwindigkeit praktisch gleich bleiben.

Tabelle 12.2: Vergleich ohne und mit Glättung

Kennwert	Glättung aus	Glättung an
Aktive Geschwindigkeitsspitze	48.772 km/h	48.584 km/h
Aktive Beschleunigungsspitze	2.982 m/s ²	0.583 m/s ²
Maximale mechanische Leistung	2515.913 W	974.146 W
Maximaler Motorstrom	61.628 A	24.843 A
End-SoC LiPo	0.0 %	0.0 %
End-SoC NMC	0.0 %	0.0 %
Minimale Spannung LiPo	31.278 V	31.292 V
Minimale Spannung NMC	31.115 V	31.097 V
Bremswiderstandsenergie	0.000 Wh	0.000 Wh

12.3 Interpretation

Beide Akkus werden im aktuellen Datensatz vollst ndig entladen. Das bedeutet nicht, dass die Akkus physikalisch identisch reagieren; vielmehr ist der Lastbedarf so hoch, dass beide Modelle vor dem Streckenende auf 0 % SoC fallen. Die NMC-Kurve liegt bei gleichem SoC typischerweise niedriger in der Spannung als die LiPo-Kurve, weshalb die minimale Spannung im NMC-Lauf niedriger ausf llt.

Der wesentliche Nutzen der Gl ttung zeigt sich in den Spitzenwerten. Sie senkt die maximal benoetigte Leistung auf rund 974 W statt 2516 W und reduziert den maximalen Motorstrom auf etwa 24,8 A statt 61,6 A. Damit wird die numerische Auswirkung von GPS-Rauschen direkt sichtbar.

Kapitel 13

Grenzen und Ausblick

Das aktuelle Modell ist bewusst schlank gehalten. Es beschreibt die im Quellcode umgesetzten Zusammenhänge nachvollziehbar, ersetzt aber keine vollständige Fahrzeugsimulation.

13.1 Aktuell implementiert

Implementiert sind GPS-basierte Kinematik, Fahrzeugkräfte, Motorstrom, Akku-SoC, Temperaturkorrektur, Luftdichteabschätzung und die Behandlung eines Bremswiderstandsfalls bei zu hohem Ladestrom.

13.2 Modellannahmen

Mehrere Vereinfachungen sind für die aktuelle Version wesentlich. Die Fahrerleistung wird nicht separat modelliert. Der Motor wird nur über eine konstante Motorkonstante beschrieben. Der Luftwiderstand nutzt eine einfache quadratische Formel. Die thermische Akkuphysik wird über lineare Korrekturfaktoren angenähert. GPS-Höhe und GPS-Position werden direkt als Eingangsrealität akzeptiert, obwohl beide Messgrößen verrauscht sein können.

13.3 Mögliche Erweiterungen

Naheliegende Erweiterungen wären ein Windmodell, ein detaillierteres Motorwirkungsgradmodell, eine echte Rekuperationslogik mit begrenzter Ladeleistung, eine mechanische Fahrerleistung, eine Validierung gegen reale Sensordaten sowie eine ausgereifere Filterparametrierung. Auch die Akkumodelle könnten um temperatur- und lastabhängige Kennlinien erweitert werden.

13.4 Einordnung

Der Wert des Projekts liegt nicht in maximaler Komplexität, sondern in der sauberen Kette von Daten zu physikalischer Last und anschließender Akku-Simulation. Für den vorhandenen Datensatz

ist diese Kette bereits schlussig nachvollziehbar und reproduzierbar.

Kapitel 14

Fazit

Das Abschlussprojekt zeigt, wie sich eine reale GPS-Strecke zu einer reproduzierbaren E-Bike-Energiesimulation ausbauen lässt. Die Software trennt Datenverarbeitung, Fahrzeugphysik, Motorlogik, Akkuverhalten und Visualisierung klar voneinander und dokumentiert den kompletten Ablauf bis zu Karten, Plots und Logs. Die Berichtserstellung selbst wird mit `latexmk` automatisiert [9].

Besonders wichtig ist die Polymorphie der Akkumodelle: LiPo und NMC nutzen dieselbe Simulationspipeline, unterscheiden sich aber in Kennlinie und Innenwiderstand. Zusammen mit der optionalen Geschwindigkeitsglatteung lässt sich damit gut untersuchen, wie empfindlich die Leistung und der Strom auf GPS-Rauschen und kurze Zeitintervalle reagieren.

Für den aktuellen Datensatz fällt die Fahrt energetisch so anspruchsvoll aus, dass beide Akkutypen vor Streckenende auf 0 % SoC fallen. Genau diese reale Grenze macht den Bericht fachlich wertvoll: Die Simulation liefert nicht nur Visualisierungen, sondern auch eine klare Aussage über das Zusammenspiel von Strecke, Modell und Energiespeicher.

Literatur

- [1] *C-MCI-LaTeX-Class-mcidoc*, GitHub repository, Git commit 735b49b5686375e546b5f7329eabcbc99a2001b8.
besucht am 23. Juli 2026. Adresse: <https://github.com/dTmC0945/C-MCI-LaTeX-Class-mcidoc>
- [2] *Abschlussprojekt: E-Bike Route Simulation*, GitHub repository, Git commit b95e7402735c1f24ae2ebe0e4d1b78d8bc875c
besucht am 23. Juli 2026. Adresse: <https://github.com/franekwoz/Abschlussprojekt>
- [3] „pandas documentation,“ besucht am 23. Juli 2026. Adresse: <https://pandas.pydata.org/docs/>
- [4] „PyYAML documentation,“ besucht am 23. Juli 2026. Adresse: <https://pyyaml.org/wiki/PyYAMLDocumentation>
- [5] „Drag Equation,“ besucht am 23. Juli 2026. Adresse: <https://www.grc.nasa.gov/www/k-12/airplane/drageq.html>
- [6] „NumPy documentation,“ besucht am 23. Juli 2026. Adresse: <https://numpy.org/doc/stable/>
- [7] „Folium documentation,“ besucht am 23. Juli 2026. Adresse: <https://python-visualization.github.io/folium/>
- [8] „Matplotlib documentation,“ besucht am 23. Juli 2026. Adresse: <https://matplotlib.org/stable/>
- [9] „latexmk documentation,“ besucht am 23. Juli 2026. Adresse: <https://ctan.org/pkg/latexmk>